Presentación



Nombre: Cristian Junior Lorenzo Abud

Matricula: 100620581

Tema: Actividad 6.1 - Realizar un resumen de XML y su uso. Coloque ejemplos

Asignatura: Lab Lenguaje de Program III

Maestro: Marlenis Concepción

Índice

Introducción 2
Desarrollo 3
1. ¿Qué es AJAX? 4
2. ¿Cuál es el propósito de AJAX? 4
3. ¿Cómo funciona AJAX en una página web? 4
4. Relación entre AJAX, JavaScript, el navegador y el servidor 5
5. Ventajas y desventajas del uso de AJAX 5
6. Usos actuales de AJAX 6
7. Diferencia entre una página web tradicional y una página que utiliza AJAX 6
8. Ejemplo sencillo de funcionamiento de AJAX 6
Conclusión 8
Referencias Bibliográficas 9

Introducción

Cuando empezamos a navegar por internet o a dar nuestros primeros pasos en el desarrollo web, es muy común no detenerse a pensar en todo lo que ocurre detrás de cámaras con cada clic que damos. Si miramos hacia atrás, la experiencia en la web hace unos años era bastante distinta y, siendo honestos, un poco frustrante: cada vez que queríamos enviar un formulario, buscar un dato o cambiar una pequeña sección de la pantalla, el navegador se veía obligado a recargar la página completa.

Este proceso no solo hacía que la navegación fuera lenta y pesada, sino que también interrumpía constantemente la experiencia del usuario, provocando esos molestos parpadeos en la pantalla y un consumo innecesario de datos, ya que el servidor tenía que volver a enviar toda la estructura HTML, los estilos CSS y las imágenes, incluso si solo habíamos cambiado una sola palabra.

Fue precisamente para resolver este problema que la comunidad de desarrollo buscó una forma de hacer la web más fluida, y ahí es donde entra en juego AJAX (Asynchronous JavaScript and XML). Al adentrarme en la investigación para este trabajo, me llamó mucho la atención descubrir que AJAX no es un lenguaje de programación nuevo ni una tecnología mágica inventada de la nada.

Más bien, es una metodología muy inteligente que combina herramientas que ya existían en los navegadores, como JavaScript y el objeto XMLHttpRequest, para lograr que la comunicación con el servidor se realice en segundo plano, de forma asíncrona. Me pareció fascinante entender que, aunque su nombre menciona "XML", en la práctica actual casi siempre se usa JSON, lo que demuestra cómo las tecnologías evolucionan y se adaptan a las necesidades reales de los desarrolladores.

El propósito de esta investigación va más allá de definir un concepto técnico; busca desglosar de manera clara y paso a paso cómo funciona AJAX, cuál es su verdadero propósito y por qué se ha convertido en la columna vertebral de las aplicaciones web modernas. A lo largo del desarrollo, explicaremos la relación de trabajo entre el navegador, JavaScript y el servidor, y analizaremos con criterio tanto sus ventajas como las limitaciones que debemos tener en cuenta al implementarlo.

Además, incluiremos ejemplos de la vida real que usamos a diario sin darnos cuenta, y un fragmento de código práctico para aterrizar la teoría. Al finalizar este documento, quedará claro que comprender AJAX no es solo un requisito académico, sino una pieza fundamental para cualquier estudiante que aspire a crear experiencias web dinámicas, eficientes y de alta calidad.

Desarrollo

1. ¿Qué es AJAX?

AJAX son las siglas de *Asynchronous JavaScript and XML* (JavaScript y XML asíncronos). No se trata de un nuevo lenguaje de programación, sino de una combinación de tecnologías existentes que trabajan en conjunto para permitir la comunicación asíncrona entre el cliente y el servidor. Según W3Schools AJAX utiliza el objeto XMLHttpRequest integrado en el navegador para intercambiar datos con un servidor web en segundo plano, junto con JavaScript y el Modelo de Objetos del Documento (DOM) para mostrar o utilizar esos datos. Aunque su nombre incluye "XML", en la actualidad es igualmente común (y a menudo preferible) transportar datos en formato JSON o texto plano.

2. ¿Cuál es el propósito de AJAX?

El propósito principal de AJAX es mejorar la experiencia del usuario (UX) al permitir que las páginas web se actualicen de manera asíncrona. Esto significa que es posible solicitar, enviar y recibir datos del servidor sin necesidad de recargar la página completa. Sus objetivos específicos incluyen:

- Reducir la latencia percibida por el usuario.
- Disminuir el consumo de ancho de banda, ya que solo se transfiere la información estrictamente necesaria.

- Crear interfaces de usuario más receptivas, similares a las de las aplicaciones de escritorio.

3. ¿Cómo funciona AJAX en una página web?

El ciclo de vida de una solicitud AJAX sigue una secuencia lógica de pasos (W3Schools, s.f.):

1. Ocurre un evento en la página web (por ejemplo, el usuario hace clic en un botón o la página termina de cargar).
2. JavaScript crea un objeto XMLHttpRequest (o utiliza la API moderna fetch).
3. Este objeto envía una solicitud HTTP al servidor web.
4. El servidor procesa la solicitud (consultando una base de datos, ejecutando lógica de negocio, etc.).
5. El servidor envía una respuesta (datos) de vuelta a la página web.
6. JavaScript lee la respuesta recibida.
7. JavaScript ejecuta la acción adecuada, como actualizar una parte específica del DOM (la página web) con los nuevos datos.

4. Relación entre AJAX, JavaScript, el navegador y el servidor

Estos cuatro elementos forman un ecosistema interdependiente:

- **JavaScript:** Actúa como el "director de orquesta". Inicia la solicitud, maneja la asincronía (mediante *callbacks*, promesas o async/await) y manipula el DOM para reflejar los cambios.
- **El Navegador:** Proporciona las APIs nativas necesarias para la comunicación en red (como XMLHttpRequest o Fetch API) y es responsable de renderizar el HTML y ejecutar el código JavaScript.
- **El Servidor:** Recibe la solicitud HTTP, la procesa de manera independiente a la renderización de la interfaz y devuelve únicamente los datos solicitados (generalmente en JSON), sin envolverlos en una plantilla HTML completa.
- **AJAX:** Es el concepto o paradigma que une a los tres anteriores, definiendo el flujo de comunicación asíncrona entre ellos.

5. Ventajas y desventajas del uso de AJAX

Ventajas:

- **Mejor experiencia de usuario:** Las interacciones son fluidas y sin interrupciones por recargas de página.
- **Eficiencia en el ancho de banda:** Solo se transfieren datos, no toda la estructura HTML, CSS y JS de la página.
- **Respuesta en tiempo real:** Permite funcionalidades como validaciones instantáneas o notificaciones en vivo.

Desventajas:

- **Dependencia de JavaScript:** Si el usuario desactiva JavaScript en su navegador, la funcionalidad AJAX dejará de operar.
- **Complejidad en la depuración:** El flujo asíncrono puede ser más difícil de rastrear y depurar que el flujo síncrono tradicional.
- **Problemas de SEO y navegación:** Históricamente, los motores de búsqueda tenían dificultades para indexar contenido cargado dinámicamente, y el botón "Atrás" del navegador no siempre funciona como el usuario espera (aunque la API History ha mitigado esto en gran medida).

6. Usos actuales de AJAX

AJAX es la base sobre la que se construyen muchas funcionalidades web modernas:

- **Formularios y validaciones:** Comprobar si un nombre de usuario ya está registrado mientras el usuario escribe, sin enviar el formulario completo.
- **Búsquedas predictivas (Autocomplete):** Mostrar sugerencias de búsqueda en tiempo real (ej. la barra de búsqueda de Google).
- **Carga de datos (Infinite Scroll):** Cargar más publicaciones o productos a medida que el usuario se desplaza hacia abajo en la página (ej. redes sociales).
- **Aplicaciones de una sola página (SPA):** Frameworks modernos como React, Angular o Vue.js utilizan principios AJAX (a través de fetch o librerías como Axios) para cargar toda la aplicación en una sola vez y actualizar solo los componentes necesarios.

7. Diferencia entre una página web tradicional y una página que utiliza AJAX

Característica	Página Web Tradicional (Síncrona)	Página Web con AJAX (Asíncrona)

Interacción	Cada acción del usuario envía un formulario y recarga toda la página.	Las acciones envían solicitudes en segundo plano; la página no se recarga.
Velocidad percibida	Lenta, debido a la descarga repetida de recursos (HTML, CSS, imágenes).	Rápida y fluida, solo se actualiza el fragmento de información necesario.
Carga del servidor	Alta, ya que debe generar y enviar una página HTML completa en cada solicitud.	Optimizada, ya que solo envía datos crudos (ej. JSON).
Experiencia de usuario	Fragmentada, con parpadeos o tiempos de espera en blanco.	Continua, similar a una aplicación de escritorio o móvil nativa.

8. Ejemplo sencillo de funcionamiento de AJAX

A continuación, se presenta un ejemplo básico utilizando el objeto clásico XMLHttpRequest. Este código muestra cómo solicitar un archivo de texto desde un servidor y actualizar un elemento de la página sin recargarla.

```
<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8">
    <title>Ejemplo Básico de AJAX</title>
</head>
<body>

    <div id="contenido">
        <h3>Este texto será cambiado por AJAX</h3>
        <!-- El botón activa la función loadDoc() al hacer clic -->
```

```
    <button type="button" onclick="loadDoc()">Cargar Datos del Servidor</button>
  </div>

  <script>
    function loadDoc() {
      // 1. Crear el objeto XMLHttpRequest
      var xhttp = new XMLHttpRequest();

      // 2. Definir qué hacer cuando cambie el estado de la solicitud
      xhttp.onreadystatechange = function() {
        // readyState 4 significa "solicitud completada"
        // status 200 significa "OK" (éxito)
        if (this.readyState == 4 && this.status == 200) {
          // 3. Actualizar el DOM con la respuesta del servidor
          document.getElementById("contenido").innerHTML = this.responseText;
        }
      };

      // 4. Configurar la solicitud: método GET, archivo destino, true (asíncrono)
      xhttp.open("GET", "datos_ejemplo.txt", true);

      // 5. Enviar la solicitud al servidor
      xhttp.send();
    }
  </script>

</body>
</html>
```

En el desarrollo web moderno, este mismo proceso se suele realizar de manera más limpia utilizando la API fetch o librerías como Axios, pero el principio fundamental de comunicación asíncrona permanece idéntico.

Conclusión

Al finalizar esta investigación y analizar cada uno de los componentes de AJAX, queda muy claro que no estamos hablando de una simple herramienta técnica más, sino de un verdadero cambio de paradigma en la forma en que construimos y experimentamos la web. Logró resolver uno de los problemas más grandes de los inicios de internet: la lentitud y la interrupción constante provocada por las recargas completas de la página.

Como estudiante, me pareció un "momento de claridad" entender la naturaleza asíncrona de este proceso. Antes de investigar a fondo, podía resultar confuso pensar cómo una página podía "hablar" con el servidor sin detenerse, pero al desglosar los pasos (la creación del objeto, la petición, la respuesta y la manipulación del DOM), todo cobra un sentido lógico y ordenado.

Es verdad que AJAX tiene algunos retos, como la dependencia total de JavaScript o los detalles de configuración necesarios para no afectar el SEO, pero sus beneficios en cuanto a velocidad, ahorro de ancho de banda y experiencia de usuario son indiscutibles y superan ampliamente estas limitaciones.

Además, es importante destacar que, aunque en este trabajo usamos XMLHttpRequest para el ejemplo por ser la base histórica y didáctica, el desarrollo web moderno ha evolucionado hacia el uso de la API fetch o librerías como Axios. Sin embargo, la esencia no ha cambiado: todas estas herramientas modernas siguen aplicando los mismos principios fundamentales que AJAX estableció hace años.

Por lo tanto, dominar este concepto no es algo del pasado, sino una base obligatoria para cualquier persona que quiera dedicarse al desarrollo frontend. A fin de cuentas, entender cómo fluyen los datos entre el cliente y el servidor es lo que nos permite dejar de hacer simples páginas estáticas y empezar a construir aplicaciones web robustas, dinámicas y realmente útiles para los usuarios.

Referencias Bibliográficas

- Mozilla Developer Network (MDN). (s.f.). XMLHttpRequest. Recuperado el 1 de noviembre de 2026, de https://developer.mozilla.org/es/docs/Web/API/XMLHttpRequest

- W3Schools. (s.f.). AJAX Introduction. Recuperado el 1 de noviembre de 2026, de https://www.w3schools.com/xml/ajax_intro.asp

- W3Schools. (s.f.). AJAX XML Http Request. Recuperado el 1 de noviembre de 2026, de https://www.w3schools.com/xml/ajax_http.asp